

源码安装和shell基础

一、源码安装



上节课我们学习了，rpm包以及rpm包安装（使用包管理器安装rpm包以解决依赖问题）。这次我们学习如何使用源码包在Linux上安装软件

为什么使用源码包安装软件？

- 1、想要安装的软件在网络软件仓库（如网络yum源）没有，手动使用rpm包依赖多（比如 GitHub上的项目大多数都是源码安装）
- 2、跨平台安装同一个版本的软件（源码安装不分发行版本，都可以安装）
- 3、安装的软件有一些特殊定制功能要求

(1.1) 源码安装过程

1.1.1 准备

- 安装C语言编译器

```
yum -y install gcc gcc++  
#因为源码包其实就是.C源文件，因此需要安装C语言翻译器：gcc
```

1.1.2 源码包安装

- 下载源码包

比如：wget <https://www.python.org/ftp/python/3.7.9/Python-3.7.9.tgz>

- 安装源码包依赖

源码包安装时，比较麻烦的一点，需要安装一些依赖的库，才能成功编译安装，通常项目官网会有说明（如无说明，则根据编译时报错信息进行安装即可）。比如python3.7安装需要如下库

```
yum -y install zlib-devel bzip2-devel openssl-devel ncurses-devel sqlite-
devel readline-devel tk-devel gdbm-devel db4-devel libpcap-devel xz-devel
libffi-devel
```

• 编译安装

解压源码包，并进入解压目录

```
tar -zxvf Python-3.7.9.tgz
cd Python-3.7.9
```

进入目录后，不熟悉的软件可以先查看说明，说明一般是文件名INSTALL或README的文件，打开查看说明。

```
- Developer's Guide: https://devguide.python.org/

Contributing to CPython
-----

For more complete instructions on contributing to CPython development,
see the `Developer Guide`_.

.. _Developer Guide: https://devguide.python.org/

Using Python
-----

Installable Python kits, and information about using Python, are available at
`python.org`_.

.. _python.org: https://www.python.org/

Build Instructions
-----

On Unix, Linux, BSD, macOS, and Cygwin::

    ./configure
    make
    make test
    sudo make install

This will install Python as ``python3``.
```

查看完后，进行编译前的配置准备：

输入命令：`./configure --prefix=/usr/local/[目录名]`

功能：

- 定义需要的功能选项
- 检测系统环境是否符合安装要求
- 把定义好的功能选项和检测系统环境的信息都写入Makefile文件，用于后续的编辑。

范例：

```
./configure --prefix=/usr/local/python3.7 --enable-shared
# 如果你是rpm包安装，那么是没有--enable-shared这个选项的，这会导致部分加密相关第三方库无法安装
```

configure准备没有问题后，进行编译：

编译命令：`make`

范例：

```
make -j 4 # -j 4表示使用4核CPU同时进行编译，加快编译速度
#如果编译报错，需要make clean清除之前已经编译的内容
```

执行命令后，查看最后几行有无报错

编译完后，执行编译安装（向/usr/local/[目录名]中写入）

编译安装命令：`make install`

范例：

```
make install
#编译如果没有报错，make install通常不会报错
```

执行命令后，查看最后几行有无报错

- **总结**

源码包安装，一般三部：

1.编译前准备 `./configure --prefix=/usr/local/[目录名]`

2.编译 `make`

3.编译安装 `make install`

每个步骤执行后检查有无报错，没有报错后，再执行下一步

- **RPM包和源码包选择**

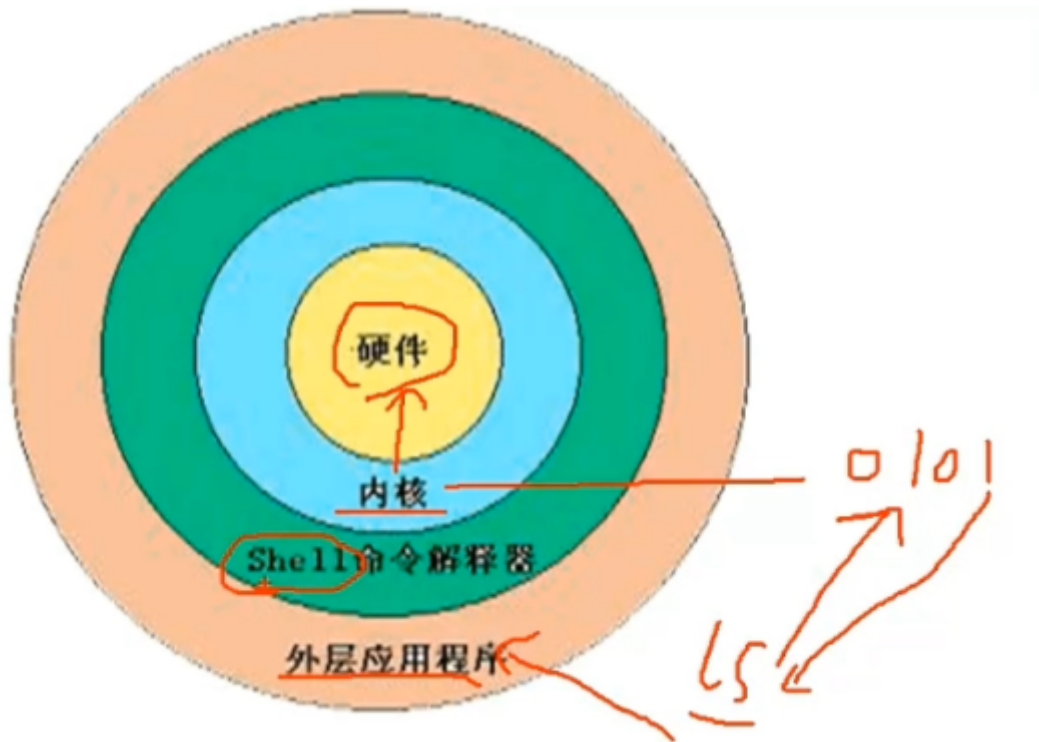
RPM包：服务器本机需要的服务（比如 gcc）

源码包：多人访问的服务，对稳定性，执行效率要求高（比如 httpd）

二、shell基础



(2.1) shell概述



shell是什么?

- 1. Shell是一个命令行解释器，它为用户提供了一个向Linux内核发送请求以便运行程序的界面系统级程序，用户可以用Shell来启动、挂起、停止甚至是编写一些程序。

问题：我们Linux登录后的，字符界面是什么？是shell命令解释器吗？---不是shell命令解释器，是shell-console一个交互式的终端，方便我们输入命令以便让shell解释器翻译成内核识别的二进制码(01010111)

我们在安全渗透过程中常说的获取目标shell是什么？---获取shell终端的执行权限

- 2. Shell还是一个功能相当强大的编程语言,易编写，易调试，灵活性较强。Shell是解释执行的脚本语言，在Shell中可以直接调用Linux系统命令

(2.2) shell分类

- **Bourne Shell:** 从1979起Unix就开始使用Bourne Shell，Bourne Shell的主文件名为sh
- **CShell:** 主要在BSD版的Unix系统中使用，其语法和C语言相类似而得名
- **Bash:** Bash与sh兼容，现在使用的Linux就是使用Bash作为用户的基本Shell。

Shell的两种主要语法类型有Bourne和C，这两种语法彼此不兼容。Bourne家族主要包括sh、ksh、Bash、psh、zsh；C家族主要包括:csh、tcsh

(2.3) bash基本功能

2.3.1 历史命令和命令补全

历史命令： `history` [选项] [历史命令保存文件]

选项：

-c: 清空历史命令

-w: 把缓存中的命令写入指定文件

默认保存目录: [家目录]/.bash_history (历史命令默认会保存1000条,可以在环境变量配置文件/etc/profile中进行修改)

历史命令的调用

- **使用上、下箭头调用以前的历史命令**
- 使用"!n"重复执行第n条历史命令
- 使用"!!"重复执行上一条命令
- 使用"!字符串"重复执行最后一条以该字符串开头的命令

历史命令调用范例

```
! 412 #执行第412行的历史命令
!! #执行上一个历史命令
! ser#执行上一个以字符串ser开头的历史命令
```

命令与文件补全

在Bash中, 命令与文件补全是非常方便与常用的功能, 我们只要在输入命令或文件时, 按"Tab"键就会自动进行补全

2.3.2 命令别名

命令别名:

```
alias [命令别名]=[命令]
```

```
alias vi=vim (命令vi相当于vim)
```

2.3.3 输入和输出重定向

- **linux标准的输入输出**

在Shell中, 0表示标准输入, 1表示标准输出, 2代表错误输出

设备	设备文件名	文件描述符	类型
键盘	/dev/stdin	0	标准输入
显示器	/dev/sdtout	1	标准输出
显示器	/dev/sdterr	2	标准错误输出

- **输出重定向**

将原本输出到屏幕的结果，输出到文件里面

类 型	符 号	作用
标准输出重定向	命令 > 文件	以覆盖的方式，把命令的正确输出输出到指定的文件或设备当中。
	+ 命令 >> 文件	以追加的方式，把命令的正确输出输出到指定的文件或设备当中。
标准错误输出重定向	错误命令 2>文件	以覆盖的方式，把命令的错误输出输出到指定的文件或设备当中。
	错误命令 2>>文件	以追加的方式，把命令的错误输出输出到指定的文件或设备当中。

同时保存正确输出和错误输出重定向

<u>正确输出和错误输出同时保存</u>	命令 > 文件 2>&1	以覆盖的方式，把正确输出和错误输出都保存到同一个文件当中。
	命令 >> 文件 2>&1	以追加的方式，把正确输出和错误输出都保存到同一个文件当中。
	命令 &>文件	以覆盖的方式，把正确输出和错误输出都保存到同一个文件当中。
	命令 &>>文件	以追加的方式，把正确输出和错误输出都保存到同一个文件当中。
	命令>>文件1 2>>文件2	把正确的输出追加到文件1中，把错误的输出追加到文件2中。

输出重定向范例：

```
# 简单的重定向
ls /home >> ls.txt
# 错误重定向
ls /opts 2>> err.log
# 统一保存
ls /tmp &>> all_ls.txt
# 分开保存
ls /var/java >> ls.txt 2>>err.log
# 将输出结果扔进黑洞，（不看输出结果）
ls &>/dev/null
# 特殊的重定向操作：快速清空文件内容
> ls.txt
```

• 输入重定向

将命令中接收输入的途径由默认的键盘改为其他文件.而不是等待从键盘输入

如何实现： 使用操作符: "<" 或 "<<", 并且常用eof来配合使用表示结束输入的内容,

输入重定向范例

```
# 输入途径为文件
grep alias < /root/.bashrc
# 通常用来，不通过交互的形式，来向文件写入内容，但是需要如cat、tee等命令的协助
# 利用输入重定向和cat命令，来向文件写入内容
# 这个方式，通常可以在shell脚本里面来创建文件同时生成内容
cat > b.txt << eof
a
b
c
eof
```

2.3.4 多命令顺序执行

主要有3种特殊符号

多命令执行符	格式	作用
;	<u>命令1</u> ; <u>命令2</u>	多个命令顺序执行， <u>命令之间没有任何逻辑联系</u>
<u>&&</u>	<u>命令1</u> && <u>命令2</u> +	逻辑与 当命令1正确执行，则命令2才会执行 当命令1执行不正确，则命令2不会执行
	命令1 命令2	逻辑或 当命令1执行不正确，则命令2才会执行 当命令1正确执行，则命令2不会执行

范例:

```
ls /home;cd /user;date
ls && echo "yes"
ls1 && echo "yes"
ls && echo "yes" || echo "no"
ls1 && echo "yes" || echo "no"
# 组合使用
ls1 &> /dev/null && echo "yes" || echo "no"
```

2.3.5 管道符

特殊字符 | 表示管道符，如果 命令1|命令2 表示命令1的正确输出作为命令2的操作对象，比如

```
df -h | grep sda
```

需要注意：管道符传递过来为文本性质并非参数,如果命令2的是对参数在操作，需要加上xargs

对比下面范例的执行结果

```
# 对文本内容进行关键字root过滤
find /etc -name passwd |grep root
# 对文件参数，进行root过滤筛选
find /etc -name passwd |xargs grep root
```

2.3.6 通配符

常用的通配符，通常有5个，如下表

通配符	作 用
<u>?</u>	匹配一个任意字符
*	匹配0个或任意多个任意字符，也就是可以匹配任何内容
[]	匹配中括号中任意一个字符。例如：[abc]代表一定匹配一个字符，或者是a，或者是b，或者是c。
[-]	匹配中括号中任意一个字符，-代表一个范围。例如：[a-z]代表匹配一个小写字母。
[^]	逻辑非，表示匹配不是中括号内的一个字符。例如：[^0-9]代表匹配一个不是数字的字符。

2.3.7 特殊符号

符 号	作 用
,,	单引号。在单引号中所有的特殊符号，如“\$”和“`”(反引号)都没有特殊含义。
""	双引号。在双引号中特殊符号都没有特殊含义，但是“\$”、“`”和“\”是例外，拥有“调用变量的值”、“引用命令”和“转义符”的特殊含义。
``	反引号。反引号括起来的内容是系统命令，在Bash中会先执行它。和\$()作用一样，不过推荐使用\$()，因为反引号非常容易看错。
\$()	和反引号作用一样，用来引用系统命令。
#	在Shell脚本中，#开头的行代表注释。
\$	用于调用变量的值，如需要调用变量name的值时，需要用\$name的方式得到变量的值。
\	转义符，跟在\之后的特殊符号将失去特殊含义，变为普通字符。如\\$将输出“\$”符号，而不当做是变量引用。

范例：

```
date="shijian"
echo "date"
echo "${date}"
echo "$date"
```


(2.4) shell脚本基础

2.4.1 变量

什么是变量

变量是计算机内存的单元，其中存放的值可以改变。当Shell脚本需要保存一些信息时，如一个文件名或是一个数字，就把它存放在一个变量中。每个变量有一个名字所以很容易引用它。使用变量可以保存有用信息，使系统获知用户相关设置，变量也可以用于保存暂时信息。

变量设置规则

- 变量名称可以由字母、数字和下划线组成但是不能以数字开头。如果变量名是 2name 则是错误的。
- 在Bash中，变量的默认类型都是字符串型如果要进行数值运算，则必修指定变量类型为数值型。

```
# 字符运算
a_num=78
b_num=22
wr_sum=${a_num}+${b_num}      # 错误的数值运算
re_sum=$(( ${a_num}+${b_num} )) # 正确的运算
echo ${wr_sum}
echo ${re_sum}
```

- 变量用等号赋值，等号左右两侧不能有空格。
- 变量的值如果有空格，需要使用单引号或双引号包括。
- 在变量的值中，可以使用“\”转义符。
- 如果需要增加变量的值，那么可以进行变量值的叠加。不过变量需要用双引号包含（"\$变量名"）或用 \${变量名} 包含。

范例

```
# 通常用在变量输出中
user_name="zhangsan"
echo hello "$user_name"
echo "hello ${user_name}" #推荐使用这种方式
```

- 如果是把命令的结果作为变量值赋予变量，则需要使用 \$() 包含命令。

```
now_time=$(date)
echo ${now_time}
```

- 自己定义的环境变量名建议大写，便于区分

```
PASS_DIR="/etc/passwd"
```

变量分类

- 用户自定义变量
- 环境变量

系统操作环境相关的数据。使用命令 env 查看（系统默认自己定义好的变量）

- **位置参数变量**

向脚本当中传递参数或数据，变量名不能自定义，变量作用是固定的。（这些变量都在脚本中进行使用）

位置参数变量	作用
<u>\$n</u>	n为数字，\$0代表命令本身，\$1-\$9代表第一到第九个参数，十以上的参数需要用大括号包含，如\${10}.
<u>\$*</u>	这个变量代表命令行中所有的参数，\$*把所有的参数看成一个整体
<u>\$@</u>	这个变量也代表命令行中所有的参数，不过\$@把每个参数区分对待
<u>\$#</u>	这个变量代表命令行中所有参数的个数

- **预定义变量**

Bash中已经定义好的变量，变量名不能自定义，变量作用也是固定的。

预定义变量	作用
<u>\$?</u>	最后一次执行的命令的返回状态。如果这个变量的值为0，证明上一个命令正确执行；如果这个变量的值为非0（具体是哪个数，由命令自己来决定），则证明上一个命令执行不正确了。
<u>\$\$</u>	当前进程的进程号（PID）
<u>\$!</u>	后台运行的最后一个进程的进程号（PID）

变量使用

2.4.2 第一个shell脚本

```
vim hello.sh
```

```
#!/bin/bash
# The first program
# Author: theon
# E-mail: theon@xxxx.net
# Version: v0.0.1
username="theon"
echo -e "Hello \e[1;33m${username}\e[0m This is first shell script'
#-e 参数表示支持反斜线控制的字符转换
# \e[1; : 开启颜色输出
# \e[0m : 关闭颜色输出
#30m=黑色, 31m=红色, 32m=绿色, 33m=黄色 34m=蓝色, 35m=洋红, 36m=青色, 37m=白色
```

执行脚本

- 赋予执行权限，直接运行

```
chmod +x hello.sh
./hello.sh
```

- 通过Bash/sh调用执行脚本

```
bash hello.sh
sh hello.sh
```

扩展：Windows编译的shell脚本放入Linux执行

Windows中字符格式与Linux不同，编译的shell脚本文件转入Linux后需要用命令转换格式：

dos2unix [shell脚本名]

(需要安装dos2unix: yum -y install dos2unix)

范例：

dos2unix Tetris.sh

扩展2：位置参数变量的输出

```
vim w.sh
```

```
#!/bin/bash
echo "位置参数为 $1 $2"
```

执行

```
sh w.sh Age 109
```

```
[root@theon ~]# vim w.sh
[root@theon ~]# sh w.sh Age 109
位置参数为 Age 109
[root@theon ~]#
```

其他



IT入门基础|网络安全运维知识

339

[下载](#)[订阅](#)[分享](#)

将节目插入微信公众号，（公众号菜单或文章中），增加曝光和播放量~ [操作指南 >](#)

零基础也能学的 IT 运维 + 网络安全入门，从常用软件、网络基础、Windows/Linux 命令学起，一步步掌握 Docker 虚拟化、Python 爬虫、Web 中间件配置，再深入渗透测试框架、PHP 漏洞分析等实战内容。全程大白话讲解，不堆砌晦涩术语，不用啃厚书、不费眼刷视频，走路、睡前、通勤时听一听，就能轻松入门 IT 运维与网络安全！

声音 (10)	评价 (0)	正序 倒序
10	掌控Linux源码安装与Shell脚本	27 3小时前
9	Linux特殊权限与运维底层逻辑	33 6天前
8	Linux权限灵魂附体	31 14天前

播客标题：掌控Linux源码安装与Shell脚本

播客内容：深入探讨Linux源码和bash核心特性

播客地址：

- 喜马拉雅：<https://www.ximalaya.com/album/118679212>
- 小宇宙：<https://namecard.xiaoyuzhoufm.com/rjrtv>

目的：辅助掌握今天的知识内容，没有时间学习的同学或者已经有基础的同学可以通过播客快速掌握大致内容

作业

尝试编写自己的第一个shell脚本

作业任务：请按照下面的要求编写自己的第一个shell脚本

- 脚本后面需要传入1个位置参数变量，变量值为用户名称

- 脚本功能之一：在 `/var/tmp` 目录下创建一个目录 `shells`，并且在目录 `shells` 下创建文件 `user_list.txt` 同时将位置参数变量和脚本执行时间（date输出的信息）同步写入到文件 `user_list.txt`
- 脚本功能之二：输出传入的位置参数变量，格式范例为：`你好 张三 你的脚本执行时间为 Thu May 22 11:50:03 CST 2025 你的姓名信息已经记录到文件 /var/tmp/shells/user_list.txt 中`，其中张三为位置参数变量，执行时间Thu May 22 11:36:21 CST 2025为系统命令调用，文件/var/tmp/user_list.txt为用户自定义的变量

任务提交：

请提交一张截图，要求如下

- 截图中需要有脚本执行操作和脚本输出（至少执行两次）
- 截图中需要有文件user_list.txt的内容（使用cat /var/tmp/shells/user_list.txt后，进行截取，至少有两条记录）

综上，范例截图如下

```
[root@theon ~]# sh work.sh 张三
你好 张三 你的脚本执行时间为 Thu May 22 13:52:26 CST 2025 你的姓名信息已经记录到文件 /var/tmp/shells/user_list.txt 中
[root@theon ~]# sh work.sh 李四
你好 李四 你的脚本执行时间为 Thu May 22 13:52:28 CST 2025 你的姓名信息已经记录到文件 /var/tmp/shells/user_list.txt 中
[root@theon ~]# cat /var/tmp/shells/user_list.txt
张三 Thu May 22 13:52:26 CST 2025
李四 Thu May 22 13:52:28 CST 2025
[root@theon ~]#
```

截图范例